COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 16, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Mendoza, Allyson Mendoza, Nathaniel	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **Parallelism and Multithreading**

1. PROCEDURE OUTPUT

Screenshot

```
#include <iostream>
#include <thread>


void print(int n, const std::string &str) {
    std::cout << "Printing integer: "<< n <<std::endl;
    std::cout<<" Printing string: " << str << std::endl;
}


int main(){

    std::thread t1(print, 10, "T.I.P.");
    t1.join();

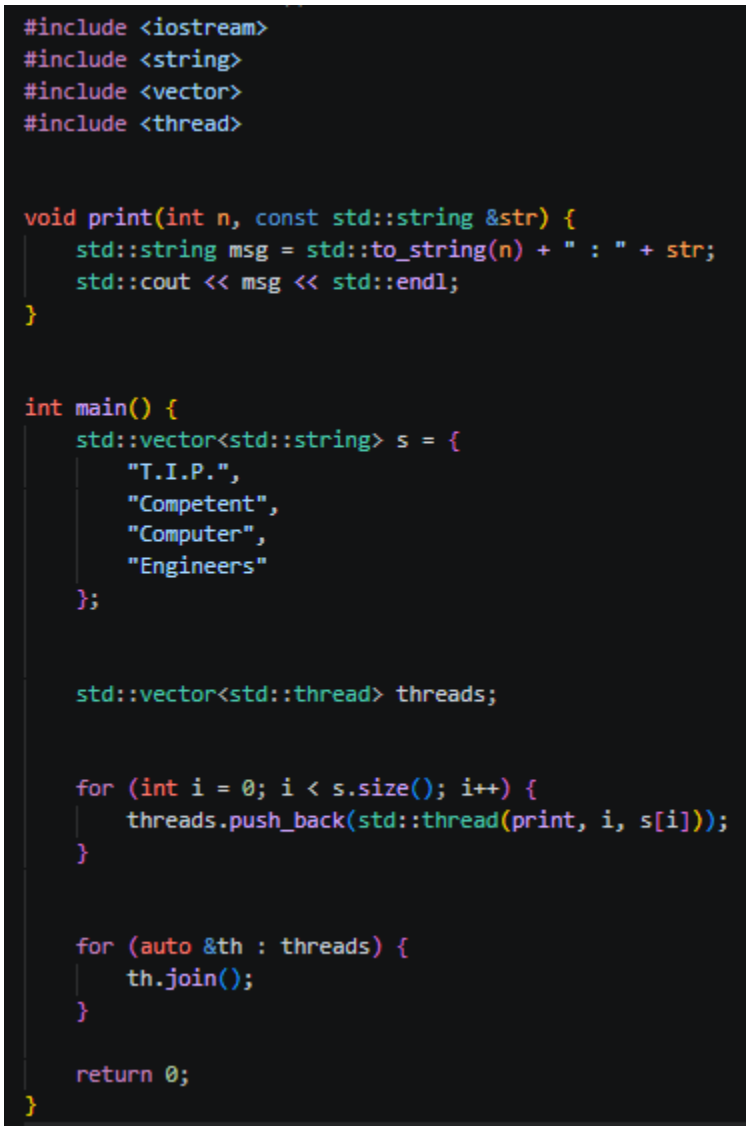
    return 0;
}
```

```
Printing integer: 10
 Printing string: T.I.P.
```

Analysis

The main program reads the printing function using its parameters from the given thread and then joins it together on print.

Table 13-1. Simple One-Threaded Example

Screenshot	
Analysis	The code prints the given dictionary where it reads it one by one and puts a gap or a reminder using ":" to show the end of the given dictionary. A loop was applied for this code approach for reading the dictionary to end.

```
#include <iostream>
#include <string>
#include <vector>
#include <thread>


void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}


int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };


    std::vector<std::thread> threads;


    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }


    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
0 : T.I.P.
1 : Competent
2 : Computer
3 : Engineers
```

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PART A

1. Write a definition of multithreading and its advantages/disadvantages.

Mendoza, A
Multithreading is a programming technique that allows a single process to execute multiple threads at the same time. Whereas each thread can perform different tasks inside the same program which can help in improving responsiveness and resource utilization. One of its advantages is the improvement of the performance and better responsiveness. While the disadvantages are complex programming and difficult debugging.
Mendoza, N
When it comes to multithreading it is beneficial when it comes to handling multiple threads specially when coming up to a certain solution. Meaning, these stay active on the program even if something is blocked or not running. Which means by default it shares the memory and improves the utilization of the program while maintaining its responsiveness. Although this type of approach consists of being highly complex which is difficult to some programmers.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

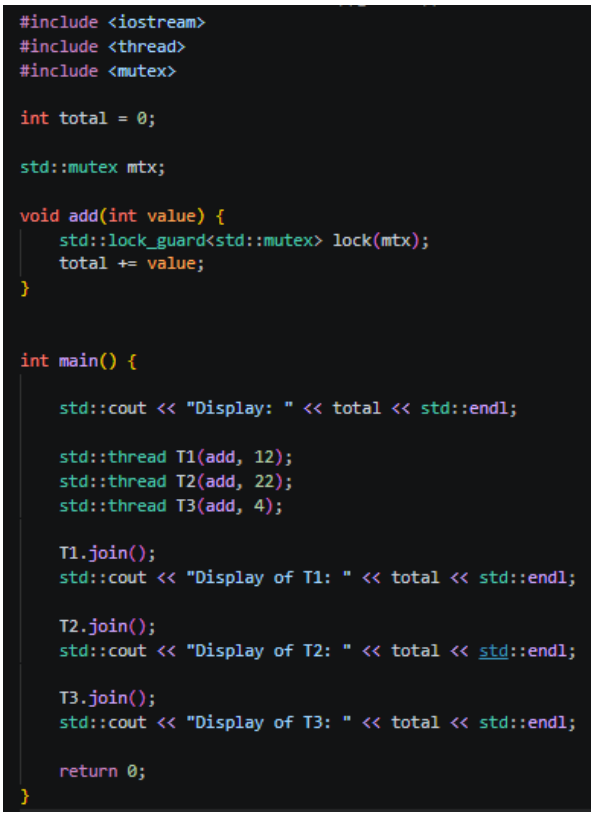
Mendoza, A
The use of multithreading is very useful as it can perform multiple tasks at the same time without making it unresponsive. The first use case is online gaming where it can maintain a smooth gaming while it processes multiple or several activities. The second is the file downloading where it can prevent the interface from freezing or crushing as the downloading processes. And lastly the third one is the video editing and streaming where it can reduce the delays and can improve the processes efficiently.
Mendoza, N
Multithreading in general optimizes software performance by allowing a single CPU process to execute multiple threads of execution concurrently. Upon searching, it shows that it can greatly handle responsive user interfaces where it can still run the program regardless of anything that's blocking this or any error on the current running program.

3. Differentiate between parallelism and concurrency.

Mendoza, A
Parallelism can perform multiple tasks at the exact same time and it can perform multiple tasks simultaneously without the reduction of its execution time. While concurrency can also perform multiple tasks but the progress can overlap and it can manage multiple tasks efficiently.

Mendoza, N
Concurrency is the manager while parallelism is the worker. Wherein it manages the multiple tasks while on the other hand it performs it.

PART B

Screenshot with Output Console	Analysis
 Display: 0 Display of T1: 38 Display of T2: 38 Display of T3: 38	The program shows how multithreading can be used to modify a shared global variable. T1, T2, and T3 are used to add up all the values to the global variable which is the total. The join() function is used to make the main thread wait for a specified thread in order to display the values. Then, mutex is used to prevent multiple threads from modifying the shared variable. Therefore, once all the threads are done, it will display the total which is 38.

```cpp
#include <iostream>
#include <thread>
#include <mutex>

int total = 0;

std::mutex mtx;

void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    total += value;
}


int main() {

    std::cout << "Display: " << total << std::endl;

    std::thread T1(add, 12);
    std::thread T2(add, 22);
    std::thread T3(add, 4);

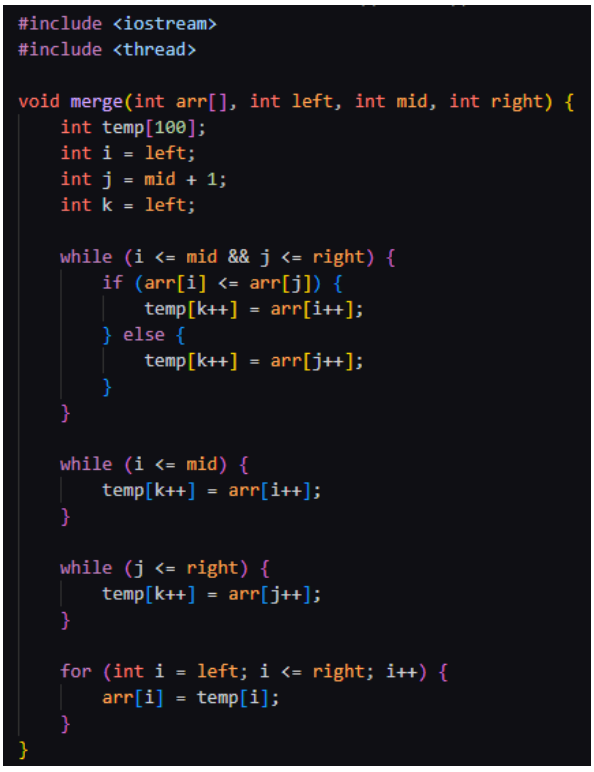
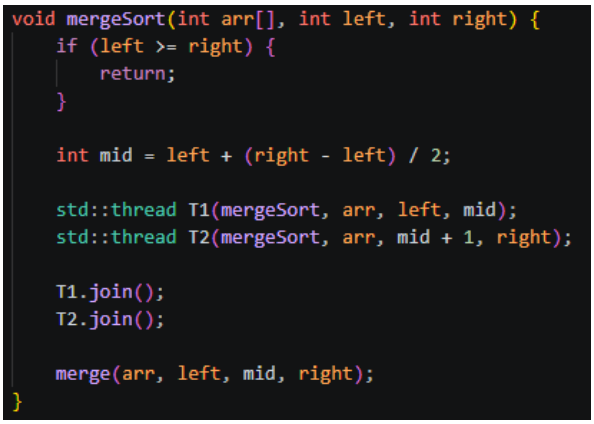
    T1.join();
    std::cout << "Display of T1: " << total << std::endl;

    T2.join();
    std::cout << "Display of T2: " << total << std::endl;

    T3.join();
    std::cout << "Display of T3: " << total << std::endl;

    return 0;
}
```

PART C

Screenshot with Output Console	Analysis
 	The program demonstrates a Merge Sort using multithreading. The array is divided into portions and separate threads sort the left and the right halves. The join() function ensures that both threads finish sorting before the merge() function combines all the portions. Therefore, after it is sorted using merge sort, the expected output confirms that the values are arranged from smallest to largest value.

```cpp
int main() {
    int arr[] = {12, 22, 4, 10, 2, 3, 7};
    int size = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Unsorted Array: ";
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;

    mergeSort(arr, 0, size - 1);

    std::cout << "Sorted Array: ";
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

```
Unsorted Array: 12 22 4 10 2 3 7
Sorted Array: 2 3 4 7 10 12 22
```

Mendoza, Allyson (CONCLUSION)
In conclusion, this hands-on activity helped me understand the use of multithreading. I learned how to perform the join() function and it should be executed either single or multi. I also learned that thread creation has both advantages and disadvantages. Overall, this hands-on activity is very informative and reliable.
Mendoza, Nathaniel (CONCLUSION)
After performing this laboratory activity, I was able to understand how multithreading and parallel algorithms work in C++. Additionally, I learned how to create and manage threads using join() and how to apply multithreading to sorting algorithms. This activity taught me that dividing tasks into smaller parts can make a program more efficient, but creating too many threads may also affect its performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.